

HabitQuest

Документація проєкту

Веб-застосунок для відстеження звичок із гейміфікацією
на основі подієво-орієнтованої архітектури (EDA)

1. Опис проєкту

1.1. Проблема та мета

Сучасна людина стикається з труднощами у формуванні та підтриманні корисних звичок і виконанні поточних завдань. Традиційні методи — паперові списки або прості нагадування — не забезпечують достатньої мотивації та зворотного зв'язку.

HabitQuest вирішує цю проблему шляхом впровадження механізмів гейміфікації: кожне виконане завдання або звичка приносить гравцю очки досвіду (XP) та ігрову валюту (монети). Накопичуючи XP, користувач підвищує рівень персонажа, а монети можна витратити в магазині на косметичні покращення — рамки для аватарки. Соціальна складова у вигляді підписок на інших гравців та перегляду їхнього прогресу створює додаткову мотивацію.

1.2. Актори системи

Система має одну роль — Гравець (користувач). Неавтентифікований відвідувач може лише зареєструватись або увійти в систему.

Актор	Опис	Основні дії
Гравець (автентифікований)	Зареєстрований користувач системи	Керування звичками та завданнями, отримання XP і монет, купівля рамок, підписки на інших гравців
Гравець (неавтентифікований)	Відвідувач без облікового запису	Реєстрація, вхід в систему

1.3. Сценарії використання (Use Cases)

№	Сценарій	Результат
UC-1	Реєстрація та вхід в систему	Створено акаунт, видано JWT токен
UC-2	Керування звичками (CRUD)	Звичка створена / оновлена / видалена
UC-3	Виконання звички або завдання	Нараховано XP та монети, записано подію в журнал
UC-4	Перегляд профілю та прогресу	Відображено статистику: XP, рівень, серія, виконані завдання

№	Сценарій	Результат
UC-5	Купівля та зміна рамки аватарки	Рамку куплено або надіто, списано монети
UC-6	Пошук та підписка на гравців	Додано підписку, відображено їхній рівень та рамку
UC-7	Перегляд журналу подій	Відображено повну історію дій користувача

2. Нефункціональні вимоги

Категорія	Вимога	Реалізація
Безпека	Автентифікація через JWT токени	jsonwebtoken, термін дії 7 днів
Безпека	Хешування паролів	bcryptjs, salt rounds = 10
Безпека	HTTP заголовки безпеки	helmet middleware
Продуктивність	Асинхронна обробка подій	RabbitMQ topic exchange, 3 черги
Надійність	Підтвердження доставки повідомлень	RabbitMQ ACK/NACK механізм
Масштабованість	Слабка зв'язність компонентів	EDA — сервіси не залежать один від одного
Зручність	REST API з документацією	Swagger UI на /api-docs
Технологічний стек	Node.js + Express + MongoDB	LTS версії, відкрита ліцензія

3. Архітектура системи

3.1. Вибір архітектурного стилю

Для проєкту обрано подієво-орієнтовану архітектуру (Event-Driven Architecture, EDA) у поєднанні з багатошаровою архітектурою (Layered Architecture). Такий підхід обґрунтований наступним:

- Слабка зв'язність — компоненти (XPService, LevelService, LogService) взаємодіють виключно через повідомлення брокера, не знаючи один про одного
- Асинхронність — після публікації події HTTP запит одразу повертає відповідь клієнту, не очікуючи завершення всіх обробників
- Розширюваність — додавання нового сервісу (наприклад, NotificationService) не потребує змін у існуючому коді
- Надійність — RabbitMQ гарантує доставку через механізм ACK/NACK та персистентні черги

3.2. Шари архітектури

Шар	Папка / Файл	Відповідальність
Presentation	frontend/	HTML/CSS/JS інтерфейс, HTTP запити до API
Controller	controllers/	Обробка запитів, валідація даних, формування відповіді
Service / Broker	services.js, broker.js	Бізнес-логіка подій, публікація та підписка через RabbitMQ
Repository	repositories/	Інкапсуляція запитів до MongoDB, патерн Repository
Data	models/	Mongoose схеми, структура колекцій

3.3. Потік подій через RabbitMQ

При виконанні звички або завдання запускається ланцюжок подій:

Крок	Подія (routing key)	Відправник	Отримувач	Дія
1	item.completed	habitController / taskController	xp_service_queue	XPService нараховує XP і монети
2	xp.updated	XPService	level_service_queue	LevelService перевіряє рівень

Крок	Подія (routing key)	Відправник	Отримувач	Дія
3	level.up	LevelService	log_service_queue	LogService записує підвищення рівня
1*	item.completed	habitController / taskController	log_service_queue	LogService записує факт виконання

4. Патерни проєктування (GoF)

4.1. Repository Pattern

Патерн Repository відокремлює логіку доступу до даних від бізнес-логіки. Контролери не взаємодіють з MongoDB напряду — всі запити до бази даних інкапсульовані у відповідних репозиторіях.

Файл репозиторію	Що інкапсулює
userRepository.js	Операції з колекцією users: пошук, оновлення, підписки, рамки
habitRepository.js	CRUD операції для колекції habits
taskRepository.js	CRUD операції для колекції tasks, включно з deleteAndReturn
eventLogRepository.js	Запис та читання журналу подій

4.2. Observer / Publisher-Subscriber Pattern

RabbitMQ реалізує патерн Publisher-Subscriber через topic exchange. Видавець публікує подію з routing key, не знаючи хто її отримає. Підписники — XPService, LevelService, LogService — незалежно реагують на відповідні події.

Черга	Routing Keys	Сервіс-підписник
xp_service_queue	item.completed	XPService — нараховує XP та монети
level_service_queue	xp.updated	LevelService — перевіряє підвищення рівня
log_service_queue	item.*, level.up, user.*, shop.*	LogService — записує всі події в eventlogs

4.3. Chain of Responsibility (Middleware)

Express middleware реалізує патерн Ланцюжок відповідальності. Кожен middleware вирішує чи продовжити обробку запиту (виклик next()) чи повернути відповідь. Файл middleware/auth.js перевіряє JWT токен до передачі запиту до контролера.

5. Аналіз принципів SOLID

Принцип	Назва	Реалізація в проєкті
S	Single Responsibility (Єдина відповідальність)	Кожен контролер відповідає лише за один ресурс: habitController — звички, shopController — магазин. Кожен сервіс має одну задачу: XPService — тільки XP, LogService — тільки логування.
O	Open/Closed (Відкритий/Закритий)	Додавання нового типу події не потребує змін у існуючих сервісах — достатньо створити новий підписник та підписати його на нову чергу RabbitMQ.
L	Liskov Substitution (Підстановка Ліскова)	Всі репозиторії мають однакову структуру методів (findAllByUser, create, update, delete), що дозволяє однаково взаємодіяти з ними з контролерів.
I	Interface Segregation (Розділення інтерфейсів)	Контролери імпортують лише потрібні функції. Наприклад, historyController використовує лише EventLogRepository.findByUser(), не маючи доступу до методів запису.
D	Dependency Inversion (Інверсія залежностей)	Контролери залежать від абстракції (функцій репозиторіїв), а не від конкретних Mongooose-моделей. Це дозволяє замінити MongoDB на іншу БД, змінивши лише репозиторії.

6. Рефакторинг та усунення проблем коду

6.1. Вихідний стан (до рефакторингу)

На початковому етапі розробки весь backend містився в одному файлі `server.js` обсягом **368 рядків**. Це є класичним прикладом антипатерну "God Object" — один файл містив одночасно підключення до бази даних, маршрути, бізнес-логіку, роботу з RabbitMQ та пряму взаємодію з Mongoose моделями.

Проблеми вихідного коду:

Проблема (Code Smell)	Опис	Де зустрічалась
God Object	Один файл виконував всі функції системи	server.js (368 рядків)
Long Method	Функції обробки маршрутів містили логіку доступу до БД	Кожен маршрут в server.js
Duplicate Code	Перевірка JWT токена повторювалась у кожному маршруті	15+ маршрутів
Direct DB Access	Контролери напряду викликали Mongoose моделі	User.findById(), Habit.find()
Magic Numbers	Значення XP та монет були розкидані по коду	Різні функції server.js

Приклад коду ДО рефакторингу (один маршрут містив всю логіку):

```
// ДО — все в одному місці в server.js
app.post('/api/habits/:id/complete', async (req, res) => {
  const header = req.headers.authorization; // JWT перевірка тут
  const token = header.split(' ')[1]; // JWT перевірка тут
  const user = jwt.verify(token, process.env.JWT_SECRET); // JWT тут
  const habit = await Habit.findOne({ // пряме звернення до БД
    _id: req.params.id, userId: user.id
  });
  await User.findByIdAndUpdate(user.id, // ще одне звернення до БД
    { $inc: { totalHabitsDone: 1 } }
  );
  const xpGain = difficulty === 'easy' ? 10 : // магічні числа тут
    difficulty === 'medium' ? 25 : 50;
  // ... ще 20 рядків логіки
});
```

6.2. Результат після рефакторингу

Код розділено на окремі шари з чіткою відповідальністю:

Було	Стало	Результат
------	-------	-----------

1 файл, 368 рядків	6 контролерів + 4 репозиторії + middleware	Кожен файл ≤ 80 рядків
JWT перевірка в кожному маршруті	middleware/auth.js — один файл	Усунено дублювання коду
Прямі виклики Mongoose	Репозиторії інкапсулюють всі запити до БД	Легко замінити БД
Магічні числа скрізь	config/constants.js — єдине місце	Легко змінити значення
Вся бізнес-логіка в маршрутах	XPService, LevelService, LogService	Кожен сервіс — одна задача

Приклад коду ПІСЛЯ рефакторингу (той самий маршрут):

```
// ПІСЛЯ — контролер лише координує, логіка в сервісах
async function complete(req, res) {
  const habit = await HabitRepository.findOne(req.params.id, req.user.id);
  if (!habit) return res.status(404).json({ message: 'Не знайдено' });
  await UserRepository.incrementStat(req.user.id, 'totalHabitsDone');
  await publish('item.completed', {
    userId: String(req.user.id),
    itemName: habit.name,
    difficulty: habit.difficulty,
    type: 'habit',
  });
  res.json({ message: 'OK' });
}
```

JWT автентифікація тепер в окремому middleware:

```
// middleware/auth.js — один раз для всіх маршрутів
function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Токен відсутній' });
  req.user = jwt.verify(token, process.env.JWT_SECRET);
  next();
}
```

Константи в окремому файлі:

```
// config/constants.js — магічні числа замінено константами
const XP_TABLE = { easy: 10, medium: 25, hard: 50 };
const COIN_TABLE = { easy: 5, medium: 12, hard: 25 };
```

6.3. Метрики покращення

Метрика	До	Після
Кількість файлів backend	1	16
Рядків у найбільшому файлі	368	~120
Дублювання JWT коду	15+ місць	1 файл
Магічні числа	Розкидані по коду	1 файл constants.js
Тестованість	Неможливо тестувати окремо	107 тестів, 82% coverage

7. Проєктування бази даних

7.1. Вибір технології

Для зберігання даних обрано MongoDB — документно-орієнтовану NoSQL базу даних. Вибір обґрунтований:

- Гнучка схема — структура документів може змінюватись без міграцій
- JSON-подібний формат — природно інтегрується з Node.js екосистемою
- Mongoose ODM — зручна робота зі схемами та валідацією
- Горизонтальне масштабування через sharding

7.2. Колекції та схеми

Колекція	Ключові поля	Призначення
users	_id, nickname, passwordHash, xp, level, coins, streak, ownedFrames, activeFrame, following, followers	Акаунти гравців, прогрес, підписки
habits	_id, userId (FK), name, difficulty, frequency, weekdays	Звички користувача
tasks	_id, userId (FK), name, difficulty, deadline	Одноразові завдання з дедлайном
eventlogs	_id, userId (FK), eventType, action, detail, xp, coins	Журнал всіх подій системи

7.3. DTO (Data Transfer Objects)

Функція `safeUser()` в `authController.js` виконує роль DTO — трансформує MongoDB документ, видаляючи чутливі поля (`passwordHash`) перед відправкою клієнту. Це захищає внутрішню структуру даних від витoku.

8. REST API та безпека

8.1. Ендпоінти API

Метод	Шлях	Опис	Auth
POST	/api/auth/register	Реєстрація нового користувача	—
POST	/api/auth/login	Вхід, отримання JWT токену	—
GET	/api/user/me	Дані поточного користувача	JWT
PUT	/api/user/profile	Оновити нікнейм, bio, аватарку	JWT
GET	/api/habits	Список звичок	JWT
POST	/api/habits	Створити звичку	JWT
PUT	/api/habits/:id	Оновити звичку	JWT
DELETE	/api/habits/:id	Видалити звичку	JWT
POST	/api/habits/:id/complete	Виконати звичку → подія в RabbitMQ	JWT
GET	/api/tasks	Список завдань	JWT
POST	/api/tasks	Створити завдання	JWT
PUT	/api/tasks/:id	Оновити завдання	JWT
DELETE	/api/tasks/:id	Видалити завдання	JWT
POST	/api/tasks/:id/complete	Виконати та видалити завдання	JWT
GET	/api/shop/frames	Список рамок магазину	JWT
POST	/api/shop/buy/:frameId	Купити рамку за монети	JWT
POST	/api/shop/equip/:frameId	Надіти / зняти рамку	JWT
GET	/api/users/search	Пошук гравців за нікнеймом	JWT
POST	/api/users/:id/follow	Підписатись / відписатись	JWT
GET	/api/users/following	Список підписок	JWT
GET	/api/history	Журнал подій	JWT

8.2. Механізми безпеки

Загроза	Захист	Реалізація
Несанкціонований доступ	JWT автентифікація	middleware/auth.js перевіряє токен на кожному захищеному маршруті
Злом паролів	bcrypt хешування	Salt rounds = 10, оригінальний пароль ніколи не зберігається
XSS, Clickjacking	HTTP Security Headers	helmet() встановлює CSP, X-Frame-Options, X-XSS-Protection
Витік даних	DTO трансформація	safeUser() видаляє passwordHash перед кожною відповіддю
Ін'єкції	Mongoose ODM	Параметризовані запити, схема валідація типів

9. Тестування

9.1. Результати покриття коду

Модуль / Файл	% Statements	% Branch	% Functions	% Lines
config/constants.js	100	100	100	100
authController.js	88.88	76.47	100	90.69
habitController.js	86.48	100	100	84.84
shopController.js	90.24	90	100	88.88
taskController.js	86.11	100	100	84.37
userController.js	85.48	89.28	100	87.03
historyController.js	83.33	100	100	83.33
userRepository.js	55.55	100	46.66	52.94
ВСЬОГО	82.07	86.66	60.37	81.34

9.2. Типи тестів

Тип	Файл	К-сть	Що перевіряє
Unit	tests/unit/constants.test.js	35	calcLevel(), XP_TABLE, COIN_TABLE, FRAMES, LEVELS
Unit	tests/unit/userRepository.test.js	11	Методи UserRepository з mock MongoDB
Integration	tests/integration/api.test.js	61	HTTP ендпоінти: статус коди, відповіді, авторизація
Всього	—	107	Покриття 82.07% Statements, 86.66% Branch

9.3. Інтеграційні тест-сценарії

Сценарій	Очікуваний результат	Результат
Реєстрація з унікальним нікнеймом	HTTP 200, повертає token і user	Pass
Реєстрація з зайнятим нікнеймом	HTTP 400, "вже зайнятий"	Pass
Вхід з невірним паролем	HTTP 400, "Невірний"	Pass
GET /api/habits без токена	HTTP 401	Pass

Сценарій	Очікуваний результат	Результат
Виконання завдання → видалення	HTTP 200, message = "OK"	Pass
Купівля рамки з недостатнім рівнем	HTTP 400, "рівень"	Pass
Купівля рамки без монет	HTTP 400, "монет"	Pass
Підписка на іншого гравця	HTTP 200, following = true	Pass
Відписка від гравця	HTTP 200, following = false	Pass

10. DevOps: Docker та CI/CD

10.1. Контейнеризація (Docker)

Застосунок повністю контейнеризовано за допомогою Docker. docker-compose.yml запускає три сервіси:

Сервіс	Docker образ	Порти	Призначення
mongodb	mongo:7	27017:27017	База даних MongoDB
rabbitmq	rabbitmq:3-management	5672, 15672	Брокер повідомлень + Management UI
backend	Dockerfile (node:20-alpine)	5000:5000	Express.js API сервер

10.2. CI/CD Pipeline (GitHub Actions)

При кожному git push GitHub Actions автоматично виконує:

- 1. Checkout — клонує репозиторій
- 2. Setup Node.js 20 — встановлює середовище виконання
- 3. npm ci — встановлює залежності з package-lock.json
- 4. npm test — запускає всі 107 тестів
- 5. Coverage report — генерує звіт покриття (лише для гілки main)
- 6. Upload artifact — зберігає звіт як артефакт GitHub Actions

Конфігурація знаходиться у файлі .github/workflows/ci.yml.

11. Структура проєкту

Файл / Папка	Призначення
frontend/	HTML/CSS/JS клієнтська частина (index.html, style.css, app.js)
backend/server.js	Точка входу: підключення, маршрути, запуск сервера
backend/broker.js	RabbitMQ обгортка: connect(), publish(), subscribe()
backend/services.js	Сервіси-підписники: XPService, LevelService, LogService
backend/models/index.js	Mongoose схеми: User, Habit, Task, EventLog
backend/config/constants.js	Константи: XP_TABLE, COIN_TABLE, LEVELS, FRAMES, calcLevel()
backend/middleware/auth.js	JWT middleware для захищених маршрутів
backend/controllers/	6 контролерів: auth, user, habit, task, shop, history
backend/repositories/	4 репозиторії: user, habit, task, eventLog
backend/tests/unit/	Unit тести: constants.test.js, userRepository.test.js (46 тестів)
backend/tests/integration/	Інтеграційні тести: api.test.js (61 тест)
backend/Dockerfile	Docker образ на базі node:20-alpine
docker-compose.yml	Оркестрація: MongoDB + RabbitMQ + Backend
.github/workflows/ci.yml	GitHub Actions CI/CD пайплайн
README.md	Документація з інструкцією запуску

12. Висновки

В результаті розроблено повноцінний веб-застосунок HabitQuest для відстеження корисних звичок із системою гейміфікації. Проєкт демонструє застосування сучасних технологій та архітектурних підходів:

- Подієво-орієнтована архітектура (EDA) з реальним брокером повідомлень RabbitMQ забезпечує асинхронну обробку та слабку зв'язність між компонентами
- Багаторівнева архітектура з розділенням Controller → Repository → Model дотримується принципів SOLID
- Застосовано три патерни проєктування GoF: Repository, Observer/Publisher-Subscriber, Chain of Responsibility
- REST API задокументовано через Swagger (OpenAPI 3.0), доступний на /api-docs
- 107 тестів (unit + integration) з покриттям 82.07% statements та 86.66% branch coverage
- Повна контейнеризація через Docker та CI/CD пайплайн через GitHub Actions
- Безпека: JWT автентифікація, bcrypt хешування, helmet HTTP заголовки, DTO трансформація

Система реалізує всі заплановані функціональні вимоги: управління звичками та завданнями, систему XP та рівнів, ігрову валюту, магазин рамок та соціальну складову у вигляді підписок між гравцями.